# Build Guide: LeaveDesk MCP Server, Step by Step

Companion to the Assignment Brief | What goes in each file, code skeletons with TODOs, and a checkpoint after every step

**How to use this guide.** Build one file at a time in the order below. Each step ends with a **Checkpoint**: run the command, compare your output with the expected output, and do not start the next step until it matches. If something breaks later, you then know which layer to look at. The skeletons show the structure and the hard parts; lines marked TODO are yours to write.

## 0. Before you start

| You need | Check with | Why |
|---|---|---|
| Python 3.10 or newer | python --version | The MCP SDK does not install on older Python |
| Node.js 18 or newer | node --version and npx --version | The MCP Inspector runs through npx |
| VS Code + GitHub Copilot, or Claude Desktop | Copilot Chat shows an Agent option | Step 4 connects a real AI app |

**Version warning.** `pip install "mcp[cli]"` now installs SDK 2.x, where `from mcp.server.fastmcp import FastMCP` fails with *No module named 'mcp.server.fastmcp'*. Always install with the pin in requirements.txt below. Check with `pip show mcp`: version must start with 1.

### 0.1 Folder structure

```
leavedesk-mcp/
├── .vscode/
│   └── mcp.json          # Step 4 - tells VS Code how to start your server
├── .gitignore            # .venv/  __pycache__/  .env
├── requirements.txt      # mcp[cli]>=1.2,<2
├── leave_data.py         # Step 1 - the pretend HR system: data + 4 plain functions (no MCP)
├── leave_server.py       # Step 2 - the MCP server: 4 tools, 1 resource, 1 prompt
├── client.py             # Step 3 - your own MCP client (no AI model)
└── notes.md              # Step 5 - your experiment observations
```

### 0.2 Environment

```
mkdir leavedesk-mcp && cd leavedesk-mcp
python -m venv .venv
source .venv/bin/activate              # Windows: .venv\Scripts\activate
echo 'mcp[cli]>=1.2,<2' > requirements.txt
pip install -r requirements.txt
pip show mcp                           # Version: 1.x   (if 2.x, the pin was not applied)
```

## 1. Step 1 - leave_data.py: the HR system, no MCP

**Job of this file:** hold all the data and four plain Python functions that answer from it. It must not import mcp. Why: an MCP tool is just a function; if the function is wrong, MCP only delivers the wrong answer faster. Testing it alone first separates Python bugs from MCP bugs. Copy the data block exactly:

```
from datetime import date, timedelta     # date = one calendar day; timedelta(days=1) = step one day forward

EMPLOYEES = {   # IDs stored in upper case so lookups can ignore how the user typed them
    "EMP-101": {"name": "Ananya Rao",  "department": "Engineering", "manager_id": "EMP-100",
                "balances": {"casual": 4, "sick": 6, "earned": 12}},
    "EMP-102": {"name": "Vikram Shah", "department": "Engineering", "manager_id": "EMP-100",
                "balances": {"casual": 1, "sick": 8, "earned": 3}},
    "EMP-103": {"name": "Fatima Khan", "department": "Finance",     "manager_id": "EMP-110",
                "balances": {"casual": 6, "sick": 2, "earned": 18}},
    "EMP-104": {"name": "Rohan Iyer",  "department": "Sales",       "manager_id": "EMP-120",
                "balances": {"casual": 0, "sick": 5, "earned": 7}},
}
LEAVE_REQUESTS = [
    {"request_id": "LR-5001", "employee_id": "EMP-101", "leave_type": "earned",
```

```
     "start_date": "2026-11-16", "end_date": "2026-11-20", "status": "Approved"},
    {"request_id": "LR-5002", "employee_id": "EMP-101", "leave_type": "casual",
     "start_date": "2026-12-04", "end_date": "2026-12-04", "status": "Pending"},
    {"request_id": "LR-5003", "employee_id": "EMP-102", "leave_type": "sick",
     "start_date": "2026-10-12", "end_date": "2026-10-13", "status": "Approved"},
    {"request_id": "LR-5004", "employee_id": "EMP-103", "leave_type": "earned",
     "start_date": "2026-12-21", "end_date": "2026-12-31", "status": "Pending"},
    {"request_id": "LR-5005", "employee_id": "EMP-104", "leave_type": "casual",
     "start_date": "2026-11-02", "end_date": "2026-11-03", "status": "Rejected"},
]
HOLIDAYS = {"2026-11-09": "Diwali Holiday", "2026-11-10": "Diwali Holiday (Day 2)",
            "2026-11-24": "Founders' Day",  "2026-12-25": "Christmas"}
LEAVE_POLICY = """Kaveri Infotech - Leave Policy (summary, FY 2026-27)
1. Casual leave: 8 days a year. At most 3 consecutive working days per request.
```

```
2. Sick leave: 10 days a year. More than 2 consecutive days needs a doctor's note.
3. Earned leave: 18 days a year. Must be applied at least 7 calendar days in advance.
4. Weekends (Saturday, Sunday) and company holidays are NOT counted as leave days.
5. Every request goes to the employee's manager for approval."""
MAX_CONSECUTIVE_CASUAL = 3     # policy rule 1 as a number the code can check
EARNED_NOTICE_DAYS = 7         # policy rule 3 as a number the code can check
```

Now the functions. Each returns a plain dict, because a dict becomes JSON that the AI can read field by field.

```
def _parse_date(text: str) -> date:
    # TODO 1: return date.fromisoformat(text.strip()); on ValueError raise
    #         ValueError(f"Invalid date '{text}'. Use the format YYYY-MM-DD, e.g. 2026-11-12.")
    ...

def get_leave_balance(employee_id: str) -> dict:
    # TODO 2: key = employee_id.strip().upper();  record = EMPLOYEES.get(key)
    # TODO 3: if record is None -> {"found": False, "employee_id": key, "message": "No employee with this ID."}
    # TODO 4: else -> {"found": True, "employee_id", "name", "department", "balances": dict(record["balances"])}
    ...

def list_leave_requests(employee_id: str, status: str = "") -> dict:
    # TODO 5: unknown employee -> same found: False dict as above
    # TODO 6: wanted = status.strip().capitalize()     ("pending" -> "Pending", "" stays "" = all)
```

```
    # TODO 7: keep requests of this employee, and of the wanted status if one was given
    # TODO 8: return {"found": True, "employee_id": key, "count": len(requests), "requests": requests}
    ...

def count_working_days(start_date: str, end_date: str) -> dict:
    # TODO 9:  parse both dates; if end < start return {"error": "end_date is before start_date."}
    # TODO 10: walk day by day from start to end (both included):
    #            day.weekday() >= 5          -> weekend, skip     (Mon=0 ... Sat=5, Sun=6)
    #            day.isoformat() in HOLIDAYS -> skip and add "2026-11-09 (Diwali Holiday)" to skipped
    #            otherwise                   -> working += 1
    # TODO 11: return {"start_date", "end_date", "calendar_days": (end-start).days + 1,
    #                  "working_days": working, "holidays_skipped": skipped}
    ...
```

```
def check_leave_eligibility(employee_id, leave_type, start_date, end_date, today: str = "") -> dict:
    # TODO 12: reuse get_leave_balance; unknown employee -> {"eligible": False, "reasons": [message]}
    # TODO 13: kind = leave_type.strip().lower(); not in balances -> eligible False, "Unknown leave type ..."
    # TODO 14: needed = count_working_days(...)["working_days"]; available = balances[kind]
    # TODO 15: reasons = []; add one sentence for EACH failed rule:
    #            needed > available                    -> "Needs 3 casual day(s) but only 1 left."
    #            casual and needed > MAX_CONSECUTIVE_CASUAL  -> "Casual leave is limited to 3 consecutive working days."
    #            earned and (start - reference).days < 7     -> "Earned leave needs 7 days' notice; this gives N."
    #          reference = _parse_date(today) if today else date.today()   (today= makes tests repeatable)
    # TODO 16: return employee_id, leave_type, working_days_needed, balance_available, holidays_skipped,
    #          "eligible": not reasons, "reasons": reasons or ["All checks passed."]
    ...

if __name__ == "__main__":
```

```
    print("1.", get_leave_balance("EMP-101"))
    print("2.", get_leave_balance("EMP-999"))
    print("3.", list_leave_requests("emp-101", "pending"))
    print("4.", count_working_days("2026-11-09", "2026-11-13"))
    print("5.", check_leave_eligibility("EMP-101", "casual", "2026-11-11", "2026-11-13", today="2026-10-28"))
    print("6.", check_leave_eligibility("EMP-102", "casual", "2026-11-11", "2026-11-13", today="2026-10-28"))
```

**Checkpoint 1.** `python leave_data.py` prints: (1) found True, balances casual 4, sick 6, earned 12; (2) found False for EMP-999; (3) count 1, request LR-5002 (lower-case input still matched); (4) calendar_days 5, **working_days 3**, holidays_skipped lists 9 and 10 Nov; (5) eligible True, needed 3, available 4; (6) eligible False, reason *Needs 3 casual day(s) but only 1 left.* Also run `count_working_days("10/11/2026", "2026-11-13")` in a Python shell: it must raise the clear YYYY-MM-DD error, not a confusing traceback.

## 2. Step 2 - leave_server.py: expose the functions over MCP

**Job of this file:** create a FastMCP server and register four tools, one resource and one prompt. Every tool is a thin wrapper: one line that calls the matching function in leave_data.py. FastMCP reads each function's **type hints** to build the input schema (which arguments, which types, which are optional) and its **docstring** to build the description the model reads. One tool is written in full as your pattern:

```
from mcp.server.fastmcp import FastMCP        # the high-level server class (SDK 1.x)
import leave_data                             # the HR system from Step 1

mcp = FastMCP("leavedesk")                    # the name clients display

@mcp.tool()                                   # register as a tool; type hints -> input schema
def get_leave_balance(employee_id: str) -> dict:
    """
    Get how many casual, sick and earned leave days an employee has left.

    Use this whenever someone asks about their remaining leave or leave balance. You do not
    know this yourself; it exists only in Kaveri Infotech's HR system.

    Args:
```

```
        employee_id: The employee's ID, for example "EMP-101".

    Returns:
        Name, department and remaining days per leave type. If the ID does not exist,
        found is False - say so instead of guessing.
    """
    return leave_data.get_leave_balance(employee_id)

# TODO 1: list_leave_requests(employee_id: str, status: str = "") -> dict
#         docstring: when to use it; status is optional: "Pending", "Approved" or "Rejected", empty = all
# TODO 2: count_working_days(start_date: str, end_date: str) -> dict
#         docstring must say: "Always use this ... Do not count the days yourself: you do not know the holidays."
# TODO 3: check_leave_eligibility(employee_id: str, leave_type: str, start_date: str, end_date: str) -> dict
#         (do NOT expose the today= argument: the server should use the real date)
```

```
@mcp.resource("policy://leave")               # the URI is the resource's address
def leave_policy() -> str:
    """Kaveri Infotech's leave policy: leave types, yearly limits and rules."""
    # TODO 4: return leave_data.LEAVE_POLICY
    ...

@mcp.prompt()                                 # arguments become fields the user fills in
def plan_my_leave(employee_id: str, start_date: str, end_date: str) -> str:
    """Plan a leave: checks balance, working days and policy, then recommends a leave type."""
    # TODO 5: return an f-string that says: "I am employee {employee_id} and I want to be on leave
    #         from {start_date} to {end_date}." then three numbered instructions to use
    #         get_leave_balance, count_working_days and check_leave_eligibility, then
    #         "tell me which leave type to apply for and why, in 4 short bullet points; mention holidays."
```

```
    ...

if __name__ == "__main__":
    mcp.run()                                 # stdio transport (default)
```

**Test it with the MCP Inspector.** From inside the project folder, with the venv active:

```
npx @modelcontextprotocol/inspector python leave_server.py
# The terminal prints:  MCP Inspector is up and running at http://localhost:6274/?MCP_PROXY_AUTH_TOKEN=...
# Open that FULL url (the token is required). Check Transport = STDIO, Command = python,
# Arguments = leave_server.py, then click Connect: the dot turns green.
```

**Checkpoint 2. Tools tab** -> List Tools shows exactly 4 tools; click each and confirm its description is your docstring and its inputs match your type hints (status is marked optional). Run `count_working_days` with 2026-11-09 and 2026-11-13: result shows working_days 3. Run `get_leave_balance` with EMP-999: found false. **Resources tab** -> List Resources shows policy://leave; clicking it shows the policy text. **Prompts tab** -> List Prompts shows plan_my_leave with three argument fields; fill them and Get Prompt shows your filled-in text. Take screenshots of the tools list and one result.

**Three classic bugs.** (a) A `print()` anywhere in leave_server.py: Connect fails or hangs, because print writes into the same stdout channel the MCP messages use. Use `import sys; print(..., file=sys.stderr)` for debugging. (b) `mcp dev leave_server.py` fails with *Unexpected token* '*E*' when uv is not installed: use the npx command above. (c) *No module named* '*mcp*': the Inspector started a Python without your venv; activate the venv first, or put the full venv Python path in Command.

## 3. Step 3 - client.py: be the client yourself

**Job of this file:** do in code exactly what you just did with Inspector buttons, with no AI model. VS Code and Claude Desktop run this same sequence internally: start the server, handshake, list, call. The MCP client library

is asynchronous, so the code runs inside `async def main()`.

```
import asyncio, json, sys
from mcp import ClientSession, StdioServerParameters      # session = sends MCP requests; params = how to start the server
from mcp.client.stdio import stdio_client                 # starts the server as a child process, connects to its stdin/stdout

SERVER = StdioServerParameters(command=sys.executable,    # the SAME Python that runs this file (has mcp installed)
                               args=["leave_server.py"])

def show_tool_result(title, result):
    print(f"\n--- {title} (isError={result.isError}) ---")    # isError=True when the tool raised an exception
    for block in result.content:                             # a list of content blocks; ours is one JSON text block
        print(json.dumps(json.loads(block.text), indent=2))

async def main():
    async with stdio_client(SERVER) as (read, write):        # 1. start leave_server.py
```

```
        async with ClientSession(read, write) as session:    # 2. wrap the two pipes in an MCP session
            await session.initialize()                       # 3. handshake: agree protocol version and features
            # TODO 1: tools = await session.list_tools();  print the list of t.name for t in tools.tools
            # TODO 2: result = await session.call_tool("count_working_days",
            #                       {"start_date": "2026-11-09", "end_date": "2026-11-13"});  show_tool_result(...)
            # TODO 3: call check_leave_eligibility for EMP-102, casual, 2026-11-11 to 2026-11-13; show it
            # TODO 4: resources = await session.list_resources(); print [str(r.uri) ...]
            #         policy = await session.read_resource("policy://leave"); print(policy.contents[0].text)
            # TODO 5: prompts = await session.list_prompts(); print names
            #         prompt = await session.get_prompt("plan_my_leave", {...3 string arguments...})
            #         print(prompt.messages[0].content.text)

if __name__ == "__main__":
    asyncio.run(main())
```

**Checkpoint 3.** `python client.py` prints, in order: TOOLS with the 4 names; count_working_days with isError=False and working_days 3; check_leave_eligibility with eligible false; RESOURCES ['policy://leave'] and the policy text; PROMPTS ['plan_my_leave'] and your filled-in prompt text. The INFO lines such as *Processing request of type ListToolsRequest* come from the server's log on stderr: they show each request arriving. Now change one date to 10/11/2026 and run again: that call shows isError=True with your YYYY-MM-DD message. Screenshot the output.

## 4. Step 4 - connect a real AI app and run the acceptance tests

**VS Code + Copilot.** Create `.vscode/mcp.json`. Use the **full path** to the venv Python, because VS Code does not activate your venv when it starts the server:

```
{
  "servers": {
    "leavedesk": {
      "type": "stdio",
      "command": "/full/path/to/leavedesk-mcp/.venv/bin/python",   // Windows: C:\\...\\.venv\\Scripts\\python.exe
      "args": ["${workspaceFolder}/leave_server.py"]
    }
  }
}
```

Click **Start** above the server entry (or Command Palette, *MCP: List Servers*). Open Copilot Chat, switch to **Agent**, click the Tools icon and confirm the 4 LeaveDesk tools are ticked. **Claude Desktop** instead: add the same command and args under `"mcpServers": {"leavedesk": {...}}` in claude_desktop_config.json, then fully quit and reopen the app. Ask each question in a **new chat**, so earlier answers do not leak in:

| ID | Ask exactly | Tool the model should call | Correct answer |
|---|---|---|---|
| A1 | How many casual leaves does EMP-101 have left? | get_leave_balance | 4 (also sick 6, earned 12) |
| A2 | Does EMP-101 have any leave requests waiting for approval? | list_leave_requests, status Pending | Yes: LR-5002, casual, 4 Dec 2026 |
| A3 | If I take 9 to 13 November 2026 off, how many leave days does it use? | count_working_days | 3 days; 9 and 10 Nov are company holidays |
| A4 | Can EMP-102 take casual leave from 11 to 13 November 2026? | check_leave_eligibility | No: needs 3 days, only 1 left |
| A5 | Can EMP-101 take casual leave from 1 to 4 December 2026? | check_leave_eligibility | No: balance (4) is enough, but casual leave is limited to 3 consecutive days |
| A6 | Can EMP-103 take earned leave from 21 to 25 December 2026? | check_leave_eligibility | Yes: uses 4 days (25 Dec is a holiday), 18 available |
| A7 | What is the leave balance of EMP-999? | get_leave_balance | No such employee; must not invent numbers |

| ID | Ask exactly | Tool the model should call | Correct answer |
|---|---|---|---|
| A8 | Attach policy://leave as context, then ask: Do I need a doctor's note for 3 days of sick leave? | none (resource read) | Yes: more than 2 consecutive sick days needs a note |
| A9 | Run the prompt /plan_my_leave with EMP-101, 2026-11-09, 2026-11-13 | get_leave_balance, count_working_days, check_leave_eligibility | A recommendation that mentions the 2 holidays and uses 3 leave days |

**Checkpoint 4.** All nine answers match the last column, and for A1 - A7 you can expand the tool call in the chat and see the arguments the model chose and the JSON your server returned. If an answer is wrong, decide which layer failed: wrong tool chosen (docstring problem), wrong arguments (date format or ID problem), or right JSON but a wrong sentence (model problem). Write that diagnosis in notes.md.

## 5. Step 5 - experiments for notes.md

| ID | Change exactly one thing | Ask | Record |
|---|---|---|---|
| E1 | Replace the docstring of count_working_days with "Counts days." Restart the server. | A3 in a new chat | Did the model still call the tool? Same answer? Restore the docstring afterwards. |
| E2 | Untick count_working_days and check_leave_eligibility in the chat's Tools list. | A3 and A5 | Does the model guess? Is the guess right? What does that tell you about putting calculations in tools? |
| E3 | No change. | What is the leave balance of Ananya Rao? | The tools take an ID, not a name. What did the model do? Which small tool would fix it? |

## 6. Troubleshooting

| Symptom | Cause | Fix |
|---|---|---|
| No module named 'mcp.server.fastmcp' | SDK 2.x installed | pip install 'mcp[cli]>=1.2,<2'; check pip show mcp |
| Inspector Connect does nothing | Opened the URL without ?MCP_PROXY_AUTH_TOKEN=... | Open the full URL printed in the terminal |
| Connect fails or hangs; JSON errors in the log | print() in leave_server.py corrupts stdout | Remove it or print to sys.stderr |
| Tools list empty in VS Code | Server crashed at start, or wrong Python in mcp.json | MCP: List Servers -> Show Output; use the full venv path |
| Model answers without calling a tool | Vague docstring, tool unticked, or not in Agent mode | Check Agent mode and the Tools list; improve the docstring |
| Code changes not picked up | The app still runs the old server process | Restart the server (VS Code: Restart; Claude Desktop: quit and reopen) |
| A6 now says not eligible | Run after 14 Dec 2026: the 7-day notice rule uses today's date | Expected behaviour; note it in notes.md |

## 7. Submission checklist

- Checkpoints 1 - 4 pass from a fresh clone: `python -m venv .venv && pip install -r requirements.txt`.
- leave_data.py has no mcp import; leave_server.py has no data and no print().
- Screenshots: Inspector tools list, one Inspector result, client.py output, chat answers for A3, A4, A7, A9.
- notes.md: E1 - E3, plus your layer diagnosis for any acceptance test that failed first time.
- .venv/ and __pycache__/ are not committed.